

System wczesnego ostrzegania na przejściu dla pieszych

Dokumentacja

1. Wstęp

1.1. Opis Projektu

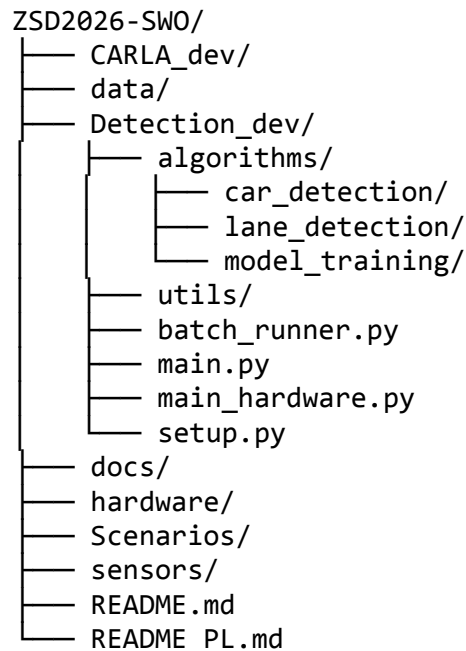
Celem projektu jest opracowanie systemu wczesnego ostrzegania na przejściu dla pieszych, który informuje o niebezpiecznych sytuacjach drogowych podnosząc alarm na 5 do 15 sekund przed dotarciem pojazdu do przejścia. Wykrywane zdarzenia niebezpieczne obejmują przekroczenie prędkości, wyprzedzanie lub zjeżdżanie z toru ruchu

1.2. Założenia projektu

- Wykrywanie zagrożeń i aktywacja alarmów na 5 do 15 sekund przed dotarciem pojazdu do strefy przejścia.
 - Monitorowanie i klasyfikacja trzech typów zdarzeń niebezpiecznych: przekroczenia prędkości, zjeżdżania z pasa ruchu oraz niebezpiecznego wyprzedzania w strefie dojazdowej do 15 metrów przed przejściem.
 - Charakterystyka drogi: Prosty, jednokierunkowy odcinek dwupasmowy o widoczności minimum 200 metrów z wyraźnym oznakowaniem poziomym.
 - Środowisko: Praca w zróżnicowanych warunkach atmosferycznych (w tym podczas opadów deszczu, mgły i gołoledzi) oraz czystego pola widzenia sensorów bez obiektów zasłaniających kadr.
 - Struktura sensoryczna: Fuzja danych ze strumienia wideo z jednej kamery o stałym polu widzenia oraz chmury punktów próbkowych radaru.
 - True Alarm Rate (TAR) - powyżej 80%
 - False Alarm Rate (FAR) - poniżej 20.00%
-

2. Struktura Projektu

2.1. Ogólna struktura projektu



2.2. Opis wybranych katalogów

2.2.1. Katalog danych



W tym katalogu znajduje się kompletny zestaw scenariuszy niebezpiecznych oraz kontrolnych: normalny ruch, kilka wariantów wyprzedzania, zjazd na pobocze i przekroczenie prędkości (szczegóły w [sekcji 6.2](#)), plik konfiguracyjny generowany przez skrypt inicjalizacyjny ([sekcja 6.1](#)) oraz używane modele: Depth-Anything-V2 (estymacja głębi) i YOLOv8 dostosowany (transfer learning) do warunków symulacyjnych CARLA.

2.2.2. Moduł detekcji

```
Detection_dev/
├── algorithms/
│   ├── car_detection/
│   ├── lane_detection/
│   │   └── lane_detector.py
│   └── model_training/
├── utils/
│   ├── alarm_manager.py
│   ├── car.py
│   ├── depth_v2.py
│   ├── radar.py
│   ├── utils.py
│   └── weather.py
├── batch_runner.py
├── main.py
├── main_hardware.py
└── setup.py
```

W tym katalogu skoncentrowana jest główna logika projektu.

- **main.py** – główny program projektu.
- **main_hardware.py** – Wariant programu głównego zoptymalizowany pod kątem uruchamiania i testów wydajnościowych na fizycznej platformie testowej.
- **batch_runner.py** – skrypt do automatycznego testowania systemu przez generowanie metryk wynikowych (omówienie metryk w [sekcji 9](#))
- **setup.py** – skrypt inicjalizacyjny, konfiguruje środowisko oraz zależności modułu detekcji ([sekcja 6.1](#)).

Podkatalog utils:

- **utils.py** – skrypt realizuje logikę wykrywania wyprzedzania ([sekcja 5.2](#)), a także funkcje pomocnicze: wizualizacja Bounding Boxów dla wykrytych przez YOLO pojazdów, generowanie wykresów porównawczych oraz zapis danych telemetrycznych obiektów do plików .csv
- **car.py** – klasa Car oraz struktury danych reprezentujących dynamiczny stan wykrytego pojazdu (trajektoria pozycji, wektor prędkości, gabaryty fizyczne, historia śledzenia). Klasa odpowiada za fuzję danych z radaru, estymację wymiarów obiektów w metrach oraz detekcję niekontrolowanego opuszczania pasa ruchu ([sekcja 5.3](#)).
- **radar.py** – moduł analizy sygnałów z sensora radarowego, realizujący filtrację, klasteryzację i geometryczną detekcję obiektów w przestrzeni 2D. ([sekcja 5](#))
- **weather.py** – integracja z zewnętrznym API pogodowym. Pobieranie odpowiednich danych dla wybranej lokalizacji, oraz wyznaczanie parametrów zależnych od pogody ([sekcja 5.1.3](#))
- **depth_v2.py** – moduł odpowiedzialny za estymację głębi. ([sekcja 5](#))

Podkatalog algorithms:

- **lane_detection (lane_detector.py)** – Algorytmy odpowiedzialne za segmentację, ekstrakcję oraz śledzenie linii wyznaczających pasy ruchu na jezdni.
- **car_detection** – pierwsze wersje implementacji YOLO
- **model_training** – Skrypty szkoleniowe: proces transfer learningu modelu YOLOv8 na danych z symulacji CARLA oraz archiwalny, pipeline generowania danych dla klasyfikatora CNN oceniającego typy pojazdów.

2.2.3. Pozostałe katalogi i pliki

- **CARLA_dev** - pliki do generacji danych w środowisku symulacyjnym
- **docs** - dokumentacja
- **hardware** - benchmarki, metryki oraz pliki konfiguracyjne związane z platformą testową
- **Scenarios, sensors** - pierwsze wersje implementacji scenariuszy i konfiguracji sensorów, zachowane jako wersje archiwalne.
- Ogólne informacje dotyczące projektu w języku polskim oraz angielskim znajdują się w plikach **README**

3. Licencje i Prawa Autorskie

Realizacja projektu wykorzystuje integrację komponentów open-source, przy czym wszystkie użyte narzędzia oraz zbiory danych są stosowane zgodnie z obowiązującymi dla nich warunkami licencyjnymi.

Narzędzie	Licencja
Python (>3.8.6)	PSF License Version 2
OpenCV (>4.5.0)	Apache License 2.0
CARLA (0.9.16)	MIT License
Ultralytics YOLO (YOLOv8)	GNU Affero General Public License v3.0
Depth-Anything-V2	Apache License 2.0
open-meteo	Creative Commons Attribution 4.0 International (CC BY 4.0) licence

Tabela 1 - licencje software

4. Stack Technologiczny

Projekt wykorzystuje następujące rozwiązania technologiczne w celu wytworzenia Systemu Wczesnego Ostrzegania:

4.1. Software:

- Python (>3.8.6) – podstawowy język programowania używany do implementacji algorytmów wykrywania i logiki systemu.
- OpenCV (>4.5.0) – używane do przetwarzania obrazu w czasie rzeczywistym.
- CARLA (0.9.16) – symulator służący do generowania danych, konfiguracji środowiska drogowego i testowania scenariuszy.
- Ultralytics YOLO (YOLOv8) – model detekcji odpowiedzialny za ciągłą identyfikację pojazdów i śledzenie trajektorii.
- Depth-Anything-V2 - model odpowiedzialny za wyznaczanie głębi obrazu w celu estymacji położenia wykrywanych pojazdów.
- Open-Meteo - API pogodowe do wyznaczania aktualnych warunków panujących na drodze w wybranej lokalizacji.

4.2. Hardware:

- Raspberry Pi 4 B – platforma testowa.
- Google Coral Edge TPU – akcelerator sprzętowy.

4.3. Wykorzystane Biblioteki:

- **Ultralytics YOLO**: realizuje detekcję obiektów klasy pojazd i dostarcza dynamiczne ramki otaczające (bounding boxes) do analizy pozycji poprzecznej i wzdłużnej.
- **torch, torchvision** – środowisko uruchomieniowe dla modeli uczenia głębokiego.
- **NumPy**: wykonuje wysokowydajne operacje macierzowe, transformacje współrzędnych 3D (korekcja obrotu radaru) oraz przekształcenia map głębi.
- **scikit-learn** (moduł sklearn.cluster.DBSCAN) – algorytm segmentacji chmury punktów.
- **OpenCV (cv2)**: obsługuje strumienie wideo, operacje na maskach graficznych, aproksymację linii pasów ruchu oraz generowanie nakładek interfejsu graficznego (GUI) dla alertów poziomu 1 i 2.

- **pandas** – szybkie wczytywanie, filtrowanie i analizowanie danych z plików CSV radaru.
 - **psutil** – monitorowanie i zbieranie próbek obciążenia procesora (cpu_percent) w celach optymalizacyjnych.
 - **matplotlib** – generowanie wykresów diagnostycznych i wizualizacji sceny 3D.
 - **requests** – obsługa zapytań HTTP do REST API stacji meteorologicznych.
-

5. Działanie Systemu Detekcji

5.1. Przekraczanie prędkości (Kod A)

5.1.1. Założenia

System identyfikuje pojazdy poruszające się z prędkością niedostosowaną do przepisów lub panujących warunków drogowych i atmosferycznych, wyliczając dynamicznie wydłużoną drogę hamowania.

5.1.2. Metoda wyznaczania prędkości pojazdu

Prędkość pobierana jest bezpośrednio z fizycznego radaru zsynchronizowanego ze strumieniem wideo. Co krok czasu chmura punktów jest filtrowana przestrzennie (maskowanie odległości wzdłużnej Y oraz wysokości Z). W celu ekstrakcji pojedynczych pojazdów, surowe punkty są skalowane wektorowo za pomocą wag i poddawane segmentacji algorytmem DBSCAN. Z każdego klastrowanego obiektu wyliczana jest stabilna średnia prędkość radialna oraz środek ciężkości. W celu eliminacji błędów montażowych, system automatycznie wylicza kąty odchylenia (*Yaw*) i obrotu (*Roll*) chmury punktów i koryguje je w przestrzeni 3D macierzą obrotu. Ponadto stałe przesunięcie między radarem a YOLO jest aproksymowane liniowo przez funkcję regresji z filtrem anomalii Z-Score.

5.1.3. Kryteria podniesienia alarmu

Alarm Poziomu 1 (Ostrzegawczy): Wyzwalany, gdy prędkość pojazdu (średnia prędkość radialna klastra oznaczona ujemnie ze względu na wektor zbliżania się) przekroczy stałą wartość progową 60 km/h.

Alarm Poziomu 2 (Krytyczny): Wyzwalany, gdy estymowana droga hamowania pojazdu nie pozwoli mu na wyhamowanie przed przejściem. System pobiera stan pogody przez API Open-Meteo, determinując mnożnik przyczepności drogi. Czas reakcji kierowcy jest zmienną nieliniową zależną

od pogody. Dynamiczna droga bezpiecznego zatrzymania (d_{stop}) dla prędkości v (w m/s) wyliczana jest jako:

$$d_{stop} = (t_{reakcji} \cdot v) + [3 \cdot (\frac{v}{10}) + (\frac{v}{10})^2] \cdot mnożnik$$

w przypadku gdy d_{stop} jest większy od dystansu do przejścia, system podnosi alarm stopnia drugiego.

5.2. Anomalia jazdy (Kod B)

5.2.1. Założenia

System monitoruje stabilność kierunku jazdy w celu wykrycia m.in. niekontrolowanego zjeżdżania z pasa ruchu.

5.2.2. Analiza toru jazdy

Linie ograniczające pasy ruchu są zdefiniowane w konfiguracji systemu za pomocą parametrów funkcji liniowej obliczanej względem osi pionowej obrazu Y (z uwagi na perspektywę kamery):

$$x = m \cdot y + b$$

Dla każdej klatki pobierane są współrzędne bounding boxa pojazdu (x_1, y_1, x_2, y_2). Następnie system wylicza teoretyczne, bezpieczne granice lewej i prawej linii dla aktualnego położenia dolnej krawędzi pojazdu (y_2):

$$\begin{aligned}x_{left\ line} &= m_{left} \cdot y_2 + b_{left} \\x_{right\ line} &= m_{right} \cdot y_2 + b_{right}\end{aligned}$$

5.2.3. Kryteria podniesienia alarmu

Alarm anomalii toru jazdy jest generowany, jeśli skrajne punkty bounding boxa pojazdu przekroczą wyznaczone granice pasów z uwzględnieniem tolerancji błędu równego 10 pikseli.

Aby zapobiec duplikowaniu alarmów w obrębie jednego manewru, system nakłada blokadę czasową wynoszącą równowartość 3 sekund czasu rzeczywistego dla danego ID obiektu.

5.3. Wyprzedzanie w bezpośredniej bliskości (Kod C)

5.3.1. Założenia

Wykrywanie manewrów wyprzedzania lub omijania na pasach oraz bezpośrednio przed nimi.

5.3.2. Analiza zdarzenia

Analiza zdarzenia: Strefa wyprzedzania inicjowana jest dynamicznie w momencie, gdy najbliższy klaster radarowy przekroczy odległość graniczną 60 metrów. Na podstawie najniższej krawędzi bounding boxa najbliższego pojazdu wyznaczana jest wirtualna linia podziału w pikselach. Obiekty znajdujące się poniżej tej linii (bliżej przejścia) są analizowane pod kątem ich wzajemnego ułożenia przestrzennego wzdłuż osi głębi. Wykorzystywany jest do tego model Depth-Anything-V2, który generuje mapę głębi klatki. Następnie pojazdom przypisywany jest relatywny ranking odległości.

5.3.3. Kryteria podniesienia alarmu

System porównuje wektory pozycji z poprzedniej oraz bieżącej klatki. Jeśli unikalne ID pojazdu, który dotychczas znajdował się dalej w rankingu geometrycznym, wyprzedzi ID pojazdu jadącego przed nim, natychmiast rejestrowana jest zmiana sekwencji i podnoszony jest alarm wyprzedzania.

6. Dane wejściowe

6.1. Skrypt Inicjalizacyjny

Podczas uruchomienia wstępnego system wykonuje następujące operacje przygotowawcze:

- Otwiera plik wideo, odczytuje parametr FPS i wylicza czas trwania pojedynczej klatki. Odczytuje opóźnienie startowe radaru z pierwszej linii pliku CSV.
- Inicjalizuje obiekt klasy radar, aplikuje maskę odległościową, nakłada symulowany szum pomiarowy oraz uruchamia procedurę automatycznego poszukiwania granic pasów ruchu w chmurze punktów.
- Wykonuje zapytanie pogodowe dla zadanej szerokości i długości geograficznej, pobierając aktualne współczynniki tarcia.
- Pobiera pierwszą klatkę referencyjną i generuje bazową mapę głębi sceny, usuwając z niej statyczne pojazdy.

- Uruchamia moduł detekcji linii w celu wyznaczenia linii prostych, oblicza szerokość drogi w pikselach i zapisuje kompletny profil konfiguracyjny do pliku konfiguracyjnego.

6.2. Dataset

Zbiór danych testowych i walidacyjnych został ustrukturyzowany w podziale na dwie główne grupy scenariuszy drogowych:

- alarm – katalog zawierający sekwencje z zarejestrowanymi incydentami i sytuacjami niebezpiecznymi.
- noalarm – katalog zawierający scenariusze kontrolne z sytuacjami, które nie powinny być wykrywane przez system

Każdy przypadek testowy składa się z pary zsynchronizowanych ze sobą plików o identycznej nazwie: nagrania wideo w formacie .mp4 oraz danych z radaru w formacie .csv, umieszczonych w odpowiadającym im podfolderze.

Nazwa pliku oraz katalogu wprost koduje typy zagrożeń występujących w danym scenariuszu za pomocą jednoznacznych oznaczeń literowych. W przypadku kumulacji zdarzeń niebezpiecznych, kody są łączone (np. 10_AC):

- **A** – przekroczenie dopuszczalnej prędkości (*speed limit exceeded*).
- **B** – niekontrolowane opuszczenie lub zjechanie z pasa ruchu (*lane departure*).
- **C** – manewr niebezpiecznego wyprzedzania w strefie przejścia (*overtaking*).

Wszystkie dane wchodzące ujednolicono pod kątem geometrii i stałych montażowych zestawu sensorów. Scenariusze zostały zarejestrowane przy zawieszeniu kamery na wysokości 5,5 metra nad poziomem jezdni oraz przy stałym kącie pochylenia osi optycznej w pionie (Pitch = -6°). Dane z radaru są pobierane dla 700 i 14000 punktów próbkowych.

6.3. Format danych wejściowych CSV

Dane z radaru dostarczane są w formacie csv uporządkowanym chronologicznie według znacznika czasu. Dane są poddawane korekcji geometrycznej.

Kolumny wejściowe (surowe dane z radaru):

- timestamp - czas rejestracji punktu od startu pomiaru (w sekundach), używany do synchronizacji z klatkami wideo.

- `radial_velocity` - prędkość radialna obiektu. W algorytmie mnożona przez -1, aby odzwierciedlała realną prędkość najazdu na przejście dla pieszych.
- `x_sensor`, `y_sensor`, `z_sensor` - surowe współrzędne obiektu (pozycja poprzeczna, odległość wzdłużna oraz wysokość) w metrach względem fizycznego sensora.

Kolumny po transformacji geometrycznej:

- `x_corrected` - pozycja poprzeczna skorygowana o automatycznie wyliczony kąt odchylenia poziomego (Yaw).
- `y_corrected` - odległość wzdłużna pojazdu do przejścia, skorygowana o kąt przechylenia bocznego (Roll).
- `z_corrected` - rzeczywista wysokość nad jezdnią. Powstaje z transformacji surowej wysokości i dodania offsetu montażu (6.0 m), co zeruje układ odniesienia na poziomie gruntu.

7. Platforma sprzętowa

7.1. Platforma testowa:

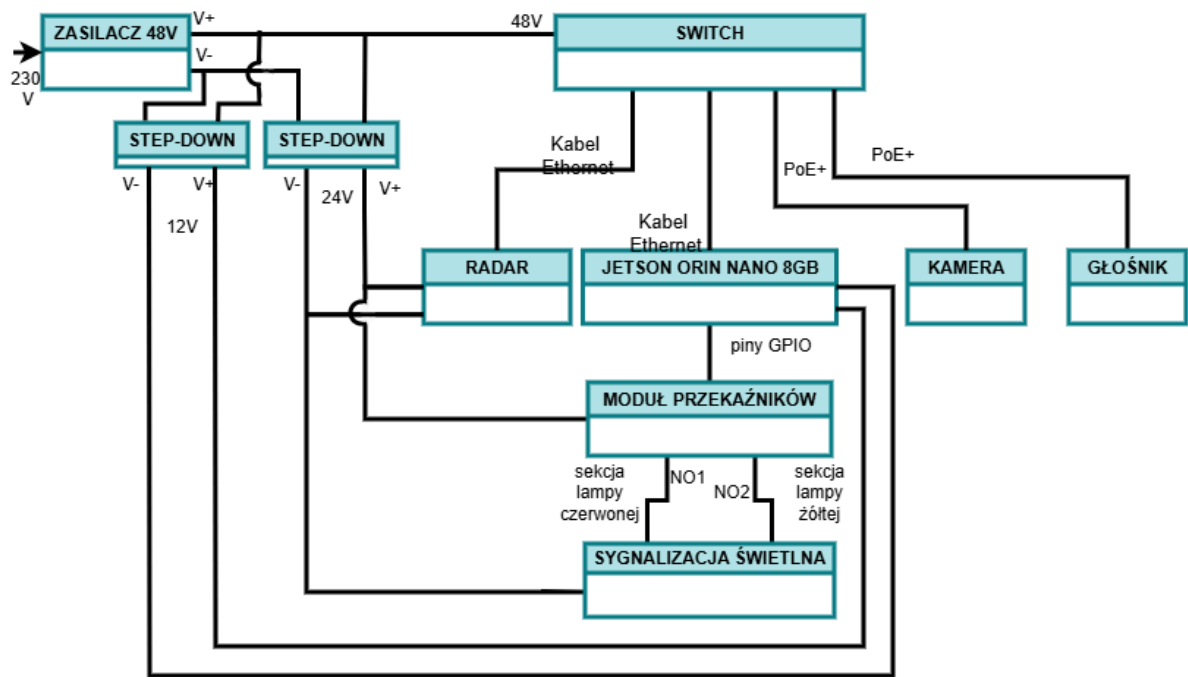
- Raspberry Pi 4 Model B
- Google Coral TPU

7.2. Proponowana platforma docelowa:

W celu eliminacji wąskich gardeł wydajnościowych platformy testowej oraz zapewnienia niezawodności w warunkach przemysłowych, dobrano komponenty docelowej architektury sprzętowej bez ograniczeń budżetowych.

Klasa komponentu	Nazwa urządzenia	Funkcja w systemie
Jednostka centralna	NVIDIA Jetson Orin Nano 8GB	Główny procesor brzegowy AI, fuzja danych i wnioskowanie sieci
Sensor radarowy	smartmicro TRUGRD LR	Long Range – ciągle skanowanie 3D drogi i pomiar prędkości radialnej
System wizyjny	AXIS P1488-LE Bullet Camera	Kamera sieciowa 4K, generowanie obrazu dla YOLO i map głębi

Tabela 2 - Wykaz komponentów platformy docelowej.



Rysunek 1 - Schemat blokowy platformy docelowej.

Opis techniczny i rola w architekturze systemu:

Jednostkę centralną systemu stanowi komputer AI NVIDIA Jetson Orin Nano 8GB. Dzięki architekturze NVIDIA Ampere z 1024 rdzeniami CUDA, moduł ten zapewnia ogromną wydajność obliczeniową do 67 TOPS, co pozwala na równoległe przetwarzanie strumienia wideo 4K oraz danych radarowych w czasie rzeczywistym.

System wizyjny oparto na profesjonalnej kamerze sieciowej AXIS P1485-LE. Urządzenie to wyposażone jest w sensor 4K oraz zmiennoogniskowy obiektyw 10.8–28.2 mm, który pozwala na precyzyjne ustawienie wąskiego pola widzenia FOV 14°. Kamera posiada zintegrowane technologie Lightfinder 2.0 oraz Forensic WDR, optymalizujące obraz pod kątem detekcji pojazdów w skrajnie trudnych warunkach oświetleniowych i pogodowych.

Sensor radarowy TRUGRD LR został wybrany ze względu na zasięg detekcji od 3 do 500 m oraz wysoką dokładność pozycjonowania obiektów. Radar pracuje w trybie ciągłym, skanując drogę i przesyłając pakiety danych o prędkości, odległości i klasyfikacji obiektów bezpośrednio do jednostki centralnej poprzez interfejs Ethernet, co gwarantuje wysoką przepustowość i stabilność transmisji.

System komunikacyjny opiera się na przemysłowym switchu PLANET ISW-514PTF-v4. Pełni on rolę centralnego węzła sieciowego, łączącego wszystkie komponenty IP. Switch wyposażony jest w funkcję Power Boost, która pozwala na zasilanie kamery oraz głośnika w standardzie PoE+ (802.3at).

System ostrzegawczy składa się z dwóch niezależnych modułów:

- Wizualnego: wykorzystującego dwukomorowy, drogowy sygnalizator LED Grupy ZIR w kolorach żółtym, ostrzegawczym oraz czerwonym, alarmowym.
- Dźwiękowego: opartego na głośniku IP AXIS C1310-E Mk II.

Moduł wykonawczy KAmoRPI PwrRELAY4 pośredniczy w sterowaniu sygnalizatorem świetlnym. Pełni on rolę izolatora galwanicznego, skutecznie oddzielając piny GPIO układu Jetson od obwodów wykonawczych 48 V. Zastosowanie optoizolacji chroni jednostkę centralną przed przepięciami oraz potencjalnymi skutkami zwarć w obwodzie zasilania lamp drogowych.

Zasilanie większości komponentów zapewnia zasilacz impulsowy MEAN WELL NDR-240-48, 5 A, 240 W. Pokrywa on zapotrzebowanie energetyczne switcha sieciowego oraz urządzeń PoE+. Zasilacz ten stanowi również źródło prądu dla radaru oraz sygnalizacji świetlnej, których napięcie robocze jest obniżane do 24 V DC za pomocą dedykowanej przetwornicy step-down. Jednostka Jetson Orin Nano zasilana dzięki przetwornicy step-down napięciem 12V.

8. Setup Guide

8.1. Instalacja środowiska

Wymagania sprzętowe:

- Urządzenie Raspberry Pi 4 B (zalecana wersja z 8 GB pamięci RAM w celu zapobiegania opóźnieniom I/O wywołanym stronicowaniem pamięci).
- Google Coral USB Accelerator podłączony do niebieskiego portu USB 3.0.

Zautomatyzowana instalacja: Wdrożenie zostało w pełni zintegrowane wewnątrz skryptu setup.sh. Skrypt samodzielnie sprawdza i w razie potrzeby pobiera środowisko Miniforge3 (Conda), instaluje biblioteki systemowe, konfiguruje sterownik o podwyższonej wydajności dla koprocatora Coral oraz ustawia profil wydajnościowy procesora.

8.2. Konfiguracja platformy testowej

Skrypt instalacyjny autonomicznie realizuje poniższe operacje konfiguracyjne w środowisku systemowym Raspberry Pi:

- **Weryfikacja i instalacja środowiska Conda (Miniforge3):** skrypt sprawdza obecność narzędzia conda w systemie. W przypadku jego braku, automatycznie pobiera oficjalny instalator dystrybucji Miniforge3 dedykowany dla architektury ARM64 (aarch64). Następnie wdraża oprogramowanie w katalogu domowym użytkownika oraz inicjalizuje powłokę Bash, gwarantując dostępność środowiska Conda podczas każdej nowej sesji terminala.
- **Aktualizacja pakietów systemowych:** inicjowane są polecenia apt-get update oraz apt-get upgrade w celu synchronizacji repozytoriów i instalacji najnowszych poprawek bezpieczeństwa systemu Debian Bookworm.
- **Instalacja wysokowydajnego sterownika Google Coral:** system weryfikuje obecność standardowego, ograniczonego wydajnościowo sterownika libedgetpu1-std (250 MHz) i w razie wykrycia – usuwa go wraz z plikami konfiguracyjnymi. W kolejnym kroku pobierana i instalowana jest zoptymalizowana wersja libedgetpu1-max, kompatybilna z Debianem Bookworm i architekturą ARM64. Działanie to odblokowuje maksymalną częstotliwość taktowania koprocatora Coral (500 MHz), co wymaga jedynie odłączenia i ponownego podłączenia go do portu USB po zakończeniu instalacji.

- **Optymalizacja wydajności procesora (CPU Governor):** domyślny zarządcą częstotliwości taktowania rdzeni (zwykle `ondemand` lub `schedutil`) jest modyfikowany na profil `performance`. Powoduje to blokadę taktowania wszystkich rdzeni procesora na maksymalnym poziomie 1.5 GHz, skutecznie eliminując opóźnienia i wahania wydajności powstające wskutek nagłych wzrostów obciążenia. W celu zachowania persystencji tego ustawienia po każdym restarcie, skrypt dopisuje odpowiednią dyrektywę do pliku konfiguracyjnego `/etc/rc.local`.
 - **Wdrożenie wirtualnego środowiska programistycznego Conda:** skrypt sprawdza, czy środowisko wirtualne o nazwie `zsd` już funkcjonuje. Jeśli nie, zostaje ono wygenerowane automatycznie na podstawie definicji zależności w pliku `environment.yaml`. W finalnej fazie skrypt dokonuje aktywacji nowo utworzonego środowiska i wykonuje aktualizację frameworka `ultralytics` do najnowszej dostępnej wersji.
-

9. Osiągnięte rezultaty

Opracowany system w pełni zrealizował pierwotne założenia projektowe, a w kluczowych metrykach efektywności i odporności na błędy znacznie je przekroczył.

- **Bezpieczeństwo i stabilność (FAR):** System wykazał się całkowitą odpornością na podnoszenie nieuzasadnionych alarmów. Margines błędu dopuszczony w pierwotnych założeniach projektowych został całkowicie zniwelowany, co potwierdza wysoką stabilność algorytmów i brak podatności na generowanie fałszywych alarmów.
- **Skuteczność wykrywania (TAR):** Opracowane algorytmy przekroczyły próg wymaganej minimalnej skuteczności, bezbłędnie identyfikując stany realnego zagrożenia na drodze.
- **System osiągnął bardzo wysoką celność klasyfikacji scenariuszy na poziomie 88% dla całej bazy zróżnicowanych nagrań walidacyjnych.**
- **Pełna Realizacja Funkcjonalności i Fuzji Sensorycznej**
- **Detekcja zagrożeń (Kody A, B, C):** Architektura systemu skutecznie integruje mechanizmy weryfikacji przekroczeń prędkości powiązanej z estymacją drogi hamowania (A), precyzyjnego monitorowania przekroczeń linii pasów ruchu z zachowaniem sztywnej tolerancji technicznej (B) oraz relatywnego pozycjonowania wzdułżnego obiektów w trakcie manewrów wyprzedzania (C).
- **Adaptacja geometryczna i sprzętowa:** Logika detekcji została optymalnie dostosowana do docelowej geometrii montażu zestawu sensorów. Wykorzystanie zaawansowanego algorytmu segmentacji przestrzennej

DBSCAN pozwoliło na stabilne wykrywanie zbliżających się klastrów pojazdów i odizolowanie ich od elementów infrastruktury.

- Dynamiczne dostosowanie do pogody: Spełniając założenie o pracy w zmiennych warunkach, do systemu wprowadzono integrację z zewnętrznym Open-Meteo API. System automatycznie pobiera kody pogodowe i dostosowuje fizykę drogi hamowania oraz czas reakcji kierowcy dla nawierzchni suchej, deszczowej oraz śliskiej (gołoledź).

Skuteczność systemu została zweryfikowana w teście wsadowym (*Batch Evaluation*) na próbie 60 nagrań testowych reprezentujących zróżnicowane scenariusze drogowe.

Metryki Wynikowe:

- Łączna liczba nagrań w bazie testowej: 60
- Liczba poprawnych klasyfikacji operacyjnych: 53
- Globalna dokładność (Accuracy): 88%
- Wskaźnik fałszywych alarmów (False Alarm Rate - FAR): 13%
- Wskaźnik pełnego wykrywania alarmów składowych (True Alarm Rate - TAR): 89%

18 nagrań sytuacji bezpiecznych: System osiągnął bardzo wysoką skuteczność (FAR = 13%). W 16 z 18 prób prawidłowo przypisano kod braku zagrożenia, pomimo występowania intensywnego szumu otoczenia (rejestracja do 16 obiektów niegroźnych, klasyfikowanych jako other w tle sceny). Filtry DBSCAN oraz maskowanie chmury punktów skutecznie odizolowały obiekty neutralne.

42 nagrania sytuacji niebezpiecznych: System poprawnie rozpoznał stan zagrożenia i komplet wymaganych kodów alarmowych w 37 z 42 nagrań (TAR ≈ 89%). W 5 przypadkach zarejestrowano rozbieżności w strukturze kodów rozpoznawanych scenariuszy.
